

Rank-One Physical Covariance Updates in the Markov Circuit

(Dated: June 1, 2026)

This note records the three algebraically equivalent forms of the rank-one Gaussian measurement update used by the CPU and GPU Markov-circuit engines: the dense Schur-complement solve, the rank-one resolvent action, and the expanded Sherman–Morrison formula. It also records the small-system survival checks used when switching the production physical covariance update to the rank-one resolvent implementation.

CONTENTS

I. Setup	1
II. Dense Schur-Complement Solver	1
III. Rank-One Resolvent Solver	2
IV. Expanded Sherman–Morrison Form	2
V. Support-Reduced Form	2
VI. Small-System Survival Checks	3
A. CPU engine	3
B. GPU class on CPU device	3

I. SETUP

Let G denote the Hermitian complex-fermion covariance on the active top layer, and let

$$P = \chi\chi^\dagger, \quad Q = \mathbb{I} - P, \quad \chi^\dagger\chi = 1. \quad (1)$$

We write a selected measurement branch using the sign

$$s = \begin{cases} +1, & \text{occupied / particle branch,} \\ -1, & \text{unoccupied / hole branch.} \end{cases} \quad (2)$$

The target covariance on the measured mode is therefore sP . The singular surface of the conditional update is

$$1 + s\chi^\dagger G\chi = 0, \quad (3)$$

which is the zero-probability conditioning surface for the selected branch.

II. DENSE SCHUR-COMPLEMENT SOLVER

The original implementation used the dense Gaussian contraction

$$G' = H_{22} - H_{21}(GH_{11} - \mathbb{I})^{-1}GH_{21}^\dagger, \quad (4)$$

with

$$(H_{11}, H_{21}, H_{22}) = \begin{cases} (-P, Q, P), & s = +1, \\ (P, Q, -P), & s = -1. \end{cases} \quad (5)$$

Equivalently,

$$\boxed{G' = sP + Q(\mathbb{I} + sGP)^{-1}GQ.} \quad (6)$$

The dense implementation evaluates Eq. (6) by solving

$$(\mathbb{I} + sGP)X = GQ. \quad (7)$$

In the production code this was previously routed through `_solve_regularized` or `_solve_regularized_batched`.

III. RANK-ONE RESOLVENT SOLVER

Since $P = \chi\chi^\dagger$,

$$GP = (G\chi)\chi^\dagger. \quad (8)$$

Thus the matrix $\mathbb{I} + sGP$ differs from the identity by rank one. Defining

$$v = G\chi, \quad d_s = 1 + s\chi^\dagger v, \quad (9)$$

the inverse action on any matrix right-hand side B is

$$\boxed{(\mathbb{I} + sGP)^{-1}B = B - \frac{s}{d_s}v(\chi^\dagger B).} \quad (10)$$

The production CPU and GPU Markov-circuit engines now use Eq. (10) for the physical covariance update. If $|d_s|$ is numerically unsafe, the implementation falls back to the same small ridged dense solve convention used elsewhere in the Markov engine.

IV. EXPANDED SHERMAN–MORRISON FORM

Substituting Eq. (10) into Eq. (6) gives the fully expanded formula

$$\boxed{G' = sP + QGQ - \frac{s}{1 + s\chi^\dagger G\chi}(QG\chi)(\chi^\dagger GQ).} \quad (11)$$

This is algebraically identical to the rank-one resolver, but it forms the outer-product correction directly. It is useful as a transparent reference implementation, while the production code keeps the resolver-action form so the support-support and support-complement right-hand sides can be handled uniformly.

V. SUPPORT-REDUCED FORM

For finite Wannier support, decompose the top-layer indices into the measured support S and complement R . With

$$G = \begin{pmatrix} G_{SS} & G_{SR} \\ G_{RS} & G_{RR} \end{pmatrix}, \quad (12)$$

and $P = \chi_S\chi_S^\dagger$ supported only on S , the rank-one action is applied only to the support block,

$$Y = (\mathbb{I} + sG_{SS}P)^{-1}G_{SR}, \quad Z = (\mathbb{I} + sG_{SS}P)^{-1}G_{SS}Q. \quad (13)$$

The updated blocks are

$$G'_{SS} = sP + QZ, \quad (14)$$

$$G'_{SR} = QY, \quad (15)$$

$$G'_{RR} = G_{RR} - sG_{RS}PY. \quad (16)$$

This is the form used by the CPU and GPU local-support kernels.

VI. SMALL-SYSTEM SURVIVAL CHECKS

The diagnostic scripts use $N_x = 4$, $N_y = 8$, 50 cycles, one sample, finite Wannier support $n_{\text{shell}} = 1$, slab-only domain-wall dynamics, and a fixed deterministic initial covariance. Blowup is defined as a nonfinite entry or $\max_{ij} |G_{ij}| > 10^{12}$.

A. CPU engine

protocol	solver	survived	first bad	final max	run max	final invol.	final spectrum
postselect	rank1	yes	—	1.000	1.105	1.818×10^{-6}	$[-1.000, 1.000]$
postselect	dense	yes	—	1.000	4.561	4.670×10^{-6}	$[-1.000, 1.000]$
postselect	Sherman	yes	—	1.000	1.879	1.299×10^{-6}	$[-1.000, 1.000]$
perfect correction	rank1	yes	—	1.000	2.132	5.701	$[-1.378, 2.511]$
perfect correction	dense	yes	—	1.000	2.132	5.701	$[-1.378, 2.511]$
perfect correction	Sherman	yes	—	1.000	2.132	5.701	$[-1.378, 2.511]$

Final-state differences relative to production rank-one were

protocol	reference	Frobenius difference	max-entry difference
postselect	dense	1.571×10^{-6}	9.370×10^{-8}
postselect	Sherman	2.614×10^{-7}	2.303×10^{-8}
perfect correction	dense	3.854×10^{-14}	4.772×10^{-15}
perfect correction	Sherman	4.618×10^{-14}	8.132×10^{-15}

B. GPU class on CPU device

CUDA was not visible in the local environment, so the GPU-class diagnostic was run on its CPU device path with the same Torch kernels and canonical `classA_U1FGTN_gpu.run_markov_circuit(...)` entry point.

protocol	solver	survived	first bad	final max	run max	final invol.	final spectrum
postselect	rank1	yes	—	1.000	1.000	2.344×10^{-6}	$[-1.000, 1.000]$
postselect	dense	yes	—	1.000	1.000	2.624×10^{-6}	$[-1.000, 1.000]$
postselect	Sherman	yes	—	1.000	1.000	3.449×10^{-6}	$[-1.000, 1.000]$
perfect correction	rank1	yes	—	1.000	1.000	2.154×10^{-13}	$[-1.000, 1.000]$
perfect correction	dense	yes	—	1.000	1.000	1.920×10^{-13}	$[-1.000, 1.000]$
perfect correction	Sherman	yes	—	1.000	1.000	2.327×10^{-13}	$[-1.000, 1.000]$

Final-state differences relative to production rank-one were

protocol	reference	Frobenius difference	max-entry difference
postselect	dense	4.717×10^{-7}	3.326×10^{-8}
postselect	Sherman	6.510×10^{-7}	6.167×10^{-8}
perfect correction	dense	5.519×10^{-14}	7.690×10^{-15}
perfect correction	Sherman	1.822×10^{-14}	2.456×10^{-15}